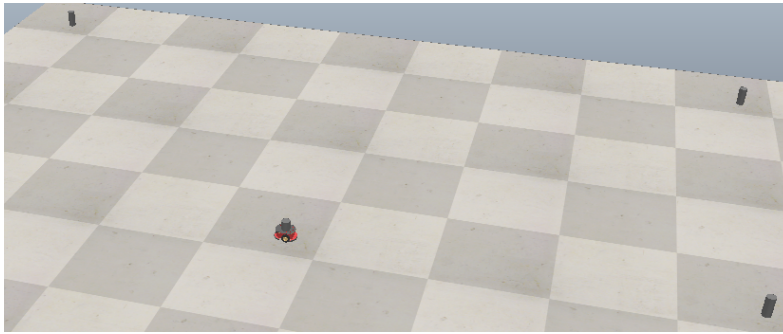


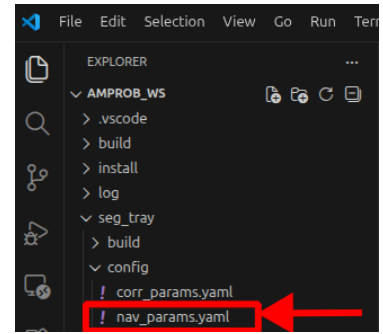
Ampliación de Robótica

Lab-MR 1-I: Seguimiento de caminos explícitos

En esta práctica se abordará el problema de seguimiento de caminos; en esta ocasión, un camino explícito definido por una serie de puntos en el entorno. Para ello, se usará un escenario en CoppeliaSIM que incluye un robot en un entorno plano, en el que se cargarán los datos del camino *waypoints* desde un archivo de parámetros *yaml* del paquete ROS2 creado para este ejercicio, *seg_tray*.



(a) Escenario de CoppeliaSIM *p2p_scene.ttt*



(b) Fichero de parámetros

Figura 1: Entorno para la práctica

En la escena configurada (ver Figura 1(a)) se especifica el camino explícito mediante una serie de puntos (*waypoints*) cargados desde un archivo de parámetros *yaml* (ver Figura 1(b)) del paquete suministrado en el Campus Virtual para esta práctica.

Modelo del vehículo

El robot que forma parte de la escena es un Pioneer3DX de guiado diferencial, con las siguientes características:

- distancia entre ruedas $2K = 0,331m$
- radio de las ruedas $R = 0,097518m$
- velocidad angular máxima $\omega_{max} = 0,79rad/s$

donde las velocidades lineal y angular, según el modelo cinemático directo, son:

$$v = \frac{(\omega_i + \omega_d) \cdot R}{2}$$

$$\omega = \frac{(\omega_d - \omega_i) \cdot R}{2K}$$

donde v y ω son las velocidades lineal y angular del vehículo, mientras que ω_i y ω_d son las velocidades angulares de la rueda izquierda y la derecha respectivamente.

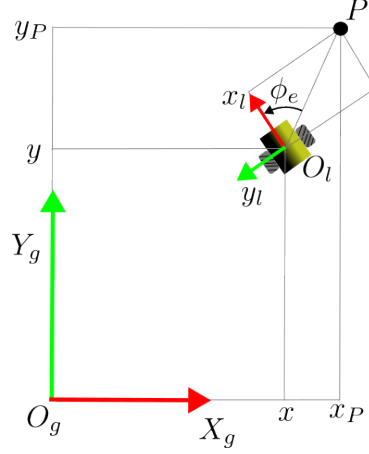


Figura 2: Sistemas de referencia. Coordenadas globales y local al vehículo

Los datos del vehículo ya se encuentran especificados en el archivo de parámetros comentado anteriormente. El control del robot se realiza con un periodo de control de $25ms$, empleando el topic `PioneerP3DX/cmd_vel`.

Navegación punto a punto

Para controlar el movimiento a través de los *waypoints*, en esta práctica se plantea utilizar la técnica de **navegación punto a punto** a una velocidad lineal de $1,2m/s$ en una superficie plana sin obstáculos. Para este movimiento se desea realizar un seguimiento con control proporcional al error de orientación:

$$\gamma = K \cdot \phi_e$$

donde γ es la curvatura, K es la ganancia proporcional y ϕ_e es el error de orientación del vehículo en un instante t . Con esta curvatura calculada en cada instante del movimiento, se pueden calcular las actuaciones sobre cada rueda del robot empleando el modelo cinemático inverso:

$$\omega_i = \frac{v \cdot (1 - K \cdot \gamma)}{R}$$

$$\omega_d = \frac{v \cdot (1 + K \cdot \gamma)}{R}$$

donde v es la velocidad lineal deseada para el movimiento del robot.

En la navegación punto a punto, el vehículo debe cambiar de objetivo cuando se encuentre a menos de $1m$ de distancia del *waypoint* que se está siguiendo hasta completar el camino. Para el cálculo del error de orientación, ϕ_e , es necesario determinar la distancia del vehículo al *waypoint* y la orientación relativa del vehículo respecto al punto objetivo (ver Figura 2).

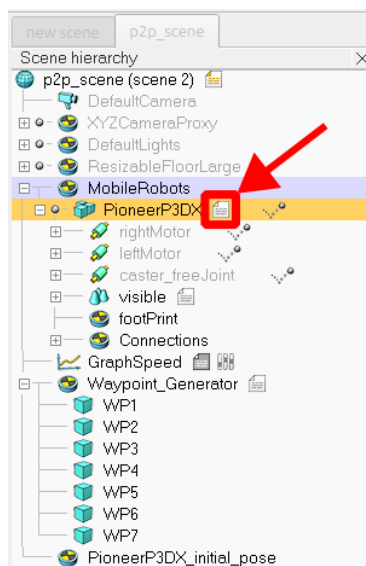
Para conseguir un cálculo directo de ϕ_e independientemente de la orientación del vehículo en coordenadas globales, es conveniente convertir las coordenadas del *waypoint* a coordenadas locales al robot, de esta forma, es posible calcular la distancia al *waypoint* y la orientación de forma más apropiada. El cambio de coordenadas globales a coordenadas locales al vehículo es:

$$x_{Pl} = \cos \phi_o \cdot (x_P - x) + \sin \phi_o \cdot (y_P - y)$$

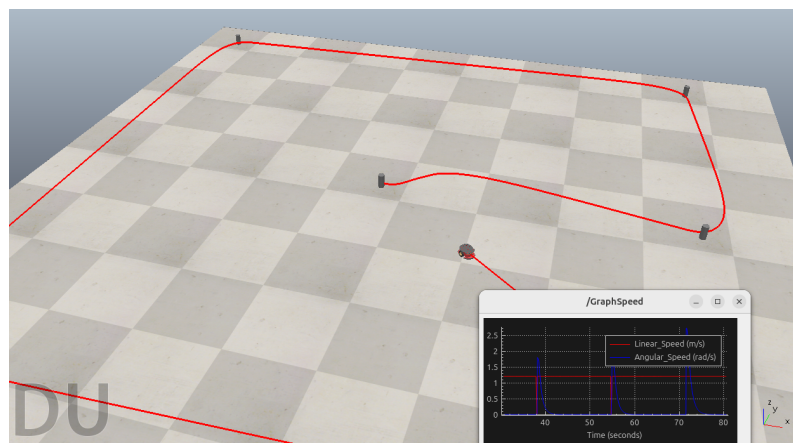
$$y_{Pl} = -\sin \phi_o \cdot (x_P - x) + \cos \phi_o \cdot (y_P - y)$$

Enunciado de la Práctica

1. Descarga y descomprime el archivo `seg_tray.zip`, que se encuentra en el Campus Virtual junto a este enunciado, en la carpeta `/home/usuario_uma/amprob_ws`. De esta forma, se ha cargado un paquete de ROS2 llamado `seg_tray`, que contiene una serie de nodos y archivos de configuración. Compila tu `workspace` para tener estos nuevos nodos disponibles.
2. Configura la escena `p2p_scene.ttt` en CoppeliaSim. Esta escena está programada para leer los *waypoints* y otros parámetros de configuración del nodo de ROS2 de esta práctica, ubicados en `seg_tray/config/nav_params.yaml`. Es necesario indicarle dónde se encuentra el archivo antes de lanzar la escena en CoppeliaSim.
 - a) Abre la escena `p2p_scene.ttt` en CoppeliaSim, ubicada en la carpeta `scenes` del mismo paquete.
 - b) Mirando la jerarquía de elementos en la escena (columna a la izquierda), haciendo doble-click sobre el icono que aparece a la derecha del elemento `PioneerP3DX` (ver figura 3(a)) se abrirá el editor de código con el script correspondiente. Edita la línea 22, que indica la ubicación del archivo de configuración, para que apunte a donde lo has descargado en tu ordenador. Por defecto, está configurado para `usuario_uma`.
 - c) Ejecuta la escena y comprueba que no hay errores. Al iniciarse, se debe cargar una serie de cilindros en la escena que representan los *waypoints* que definen el camino.
3. En el archivo con el código del nodo `nav_p2p.cpp` completa donde se indica (en el método `controlLoop()`) con las ecuaciones correspondientes para realizar la navegación punto a punto. Para ejecutar el nodo debes utilizar el archivo de lanzamiento, no el ejecutable del paquete, para que pueda encontrar el archivo de parámetros necesario para la ejecución del nodo. Para esto, debes usar: `ros2 launch seg_tray nav_p2p_launch.py`.
4. Una vez que consigas realizar la navegación punto a punto del recorrido completo (ver Figura 3(b)), modifica la ganancia y comenta cómo influye en la trayectoria realizada por el robot.



(a) Jerarquía de elementos de la escena



(b) Fichero de parámetros

Figura 3: Entorno para la práctica